

Practical 1

Create a simple HTML form and accept the user name and display the name through PHP echo statement.

Theory

HTML forms are used to collect input from users and send that data to a server for processing. A form can contain different input elements such as text boxes, radio buttons, checkboxes, dropdown menus, and submit buttons. When the user fills out the form and clicks the submit button, the data is sent to a server-side script.

PHP is a server-side scripting language that is commonly used to handle form data. It runs on the web server and processes the data before sending the result back to the user's browser. In PHP, form data can be accessed using superglobal variables such as `$_GET` and `$_POST`.

The POST method is more secure because the data is not visible in the URL. The `echo` statement in PHP is used to display output on the screen.

In this practical, a simple HTML form is created to accept a user's name. The form sends the data to a PHP script, which retrieves the value and displays it using the echo statement.

Code

index.html

```
<!DOCTYPE html>
<html>
<head>
    <title>User Form</title>
</head>
<body>
    <form action="display.php" method="post">
        Enter your name:
        <input type="text" name="username">
        <input type="submit" value="Submit">
    </form>
</body>
</html>
```

display.php

```
<?php
$name = $_POST['username'];
echo "Hello, " . $name;
?>
```

Output

If the user enters:

Himanshu

Output:

Hello, Himanshu

Practical 2

Write a PHP script to demonstrate arithmetic, comparison, and logical operators.

Theory

Operators are symbols used to perform operations on variables and values. PHP supports different types of operators such as arithmetic, comparison, and logical operators.

Arithmetic operators are used for mathematical calculations like addition, subtraction, multiplication, division, and modulus. These operators are widely used in programs that involve calculations.

Comparison operators are used to compare two values. They return either true or false depending on the result of the comparison. These are mainly used in conditional statements.

Logical operators are used to combine multiple conditions. They are useful when more than one condition needs to be checked at the same time.

This practical demonstrates how arithmetic, comparison, and logical operators work in PHP.

Code

```
<?php
$a = 10;
$b = 5;

echo "Addition: " . ($a + $b) . "<br>";
echo "Subtraction: " . ($a - $b) . "<br>";

echo "Is a greater than b? ";
var_dump($a > $b);
echo "<br>";

echo "Logical AND (a>5 && b<10): ";
var_dump($a > 5 && $b < 10);
?>
```

Output

```
Addition: 15
Subtraction: 5
Is a greater than b? true
Logical AND (a>5 && b<10): true
```

Practical 3

Write PHP script to input marks, generate result and display grade.

Theory

Conditional statements are used in programming to make decisions based on conditions. PHP provides if, else if, and else statements for decision-making.

The if statement checks a condition. If the condition is true, the block of code inside the if statement is executed. If the condition is false, the program checks the next condition using else if. If none of the conditions are true, the else block is executed.

Grading systems in schools and colleges are based on marks or percentages. Different ranges of marks correspond to different grades. For example, marks above 90 may get grade A, while marks below 60 may result in failure.

This practical demonstrates how conditional statements are used in PHP to generate grades based on marks.

Code

```
<?php
$marks = 78;

if ($marks >= 90)
    echo "Grade: A";
else if ($marks >= 75)
    echo "Grade: B";
else if ($marks >= 60)
    echo "Grade: C";
else
    echo "Grade: Fail";
?>
```

Output

Grade: B

Practical 4

Write PHP Script for addition of two 2×2 matrices.

Theory

A matrix is a rectangular arrangement of numbers in rows and columns. It is widely used in mathematics, engineering, and computer graphics. Matrix operations include addition, subtraction, and multiplication.

Matrix addition is performed by adding corresponding elements of two matrices. For matrix addition to be possible, both matrices must have the same dimensions.

In PHP, matrices can be represented using two-dimensional arrays. Nested loops are used to access each element of the matrices and perform the addition.

This practical demonstrates how to represent matrices using arrays and perform matrix addition using loops.

Code

```
<?php
$a = [[1, 2], [3, 4]];
$b = [[5, 6], [7, 8]];
$result = [];

for ($i = 0; $i < 2; $i++) {
    for ($j = 0; $j < 2; $j++) {
        $result[$i][$j] = $a[$i][$j] + $b[$i][$j];
    }
}

echo "Result Matrix:<br>";
for ($i = 0; $i < 2; $i++) {
    for ($j = 0; $j < 2; $j++) {
        echo $result[$i][$j] . " ";
    }
    echo "<br>";
}
?>
```

Output

Result Matrix:

6 8

10 12

Practical 5

Write PHP script to obtain factorial of a number using function.

Theory

A function is a block of code that performs a specific task. Functions help in organizing the program into smaller, reusable parts. They improve code readability and make maintenance easier.

The factorial of a number is the product of all positive integers from 1 up to that number. It is represented by the symbol "!". For example, $5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$.

Factorials are commonly used in mathematics, probability, and combinatorics. In PHP, functions are created using the `function` keyword and can return values using the `return` statement.

This practical demonstrates how to create a function in PHP to calculate the factorial of a number.

Code

```
<?php
function factorial($n) {
    $fact = 1;
    for ($i = 1; $i <= $n; $i++) {
        $fact *= $i;
    }
    return $fact;
}

echo "Factorial of 5 is: " . factorial(5);
?>
```

Output

Factorial of 5 is: 120

Practical 6

Write PHP script to demonstrate string, date and math functions.

Theory

PHP provides built-in functions to simplify programming tasks. These functions help in performing operations on strings, dates, and numbers.

String functions are used to manipulate text. For example, `strlen()` finds the length of a string, and `strtoupper()` converts a string to uppercase.

Date functions are used to display or manipulate date and time. The `date()` function is commonly used to display the current date in different formats.

Math functions are used to perform mathematical calculations. Functions like `sqrt()` calculate square roots, and `rand()` generates random numbers.

This practical demonstrates the use of string, date, and math functions in PHP.

Code

```
<?php
$str = "hello php";

echo "Length: " . strlen($str) . "<br>";
echo "Uppercase: " . strtoupper($str) . "<br>";

echo "Today's Date: " . date("Y-m-d") . "<br>";

echo "Square root of 25: " . sqrt(25) . "<br>";
echo "Random number: " . rand(1, 100);
?>
```

Output

```
Length: 9
Uppercase: HELLO PHP
Today's Date: 2026-02-17
Square root of 25: 5
Random number: 42
```

Practical 7

Write PHP script to demonstrate passing variables through a URL.

Theory

Data can be transferred from one webpage to another using the URL. This method is called the GET method. In this method, data is appended to the URL after a question mark.

Multiple variables can be passed using the ampersand symbol. PHP provides the `$_GET` superglobal array to access these values.

This method is simple and useful for sending small amounts of data, but it is not secure for sensitive information because the data is visible in the URL.

This practical demonstrates how to pass variables through the URL and retrieve them using PHP.

Code

link.html

```
<a href="show.php?name=Himanshu&age=20">Click Here</a>
```

show.php

```
<?php
echo "Name: " . $_GET['name'] . "<br>";
echo "Age: " . $_GET['age'];?>
```

Output

Name: Himanshu

Age: 20

Practical 8

Write PHP script to demonstrate passing variables with sessions.

Theory

Sessions are used to store user data on the server. Each user is assigned a unique session ID, which is used to access their stored data.

Sessions are commonly used in login systems, shopping carts, and other applications where user data needs to be stored temporarily.

In PHP, sessions are started using the `session_start()` function. After starting the session, variables can be stored in the `$_SESSION` array and accessed on other pages.

This practical demonstrates how to store and retrieve data using sessions.

Code

page1.php

```
<?php
session_start();
$_SESSION['username'] = "Himanshu";
echo "Session set.";
?>
```

page2.php

```
<?php
session_start();
echo "Username: " . $_SESSION['username'];
?>
```

Output

Username: Himanshu

Practical 9

Write PHP script to demonstrate passing variables with cookies.

Theory

Cookies are small files stored on the user's browser. They are used to remember user data such as login information, preferences, or visit counts.

In PHP, cookies are created using the `setcookie()` function. The cookie value can be accessed using the `$_COOKIE` array.

Cookies have an expiration time. After the expiration time, the cookie is automatically deleted from the browser.

This practical demonstrates how to create and retrieve cookies using PHP.

Code

setcookie.php

```
<?php
setcookie("user", "Himanshu", time() + 3600);
echo "Cookie set.";
?>
```

getcookie.php

```
<?php
echo "User: " . $_COOKIE['user'];
?>
```

Output

User: Himanshu

Practical 10

Write a program to keep track of how many times a visitor has loaded the page.

Theory

Websites often need to track user visits. This can be done using cookies. A cookie can store the number of visits, and each time the page loads, the number increases.

The program first checks whether the cookie already exists. If it exists, the value is increased by one. If it does not exist, the cookie is created with the value 1.

This technique is commonly used in web analytics and personalization systems.

This practical demonstrates how cookies can be used to count page visits.

Code

```
<?php
if(isset($_COOKIE['visits'])) {
    $visits = $_COOKIE['visits'] + 1;
} else {
    $visits = 1;
}

setcookie('visits', $visits, time() + 3600);

echo "Number of visits: " . $visits;
?>
```

Output

First visit:

Number of visits: 1

Next visit: Number of visits: 2

